ns_iteration

October 22, 2026

1 Newton–Schulz Iteration

Newton–Schulz (NS) is an iterative refinement scheme for multivector inverses. Given an approximate inverse $X_0 \approx A^{-1}$, each step applies:

$$X_{k+1} = X_k \cdot (2e_0 - A \cdot X_k)$$

which converges quadratically when $\|e_0 - AX_0\|$ is small.

In practice, the FLS result is already at machine precision for dimensions 6–11. For dimensions 12–13, accumulated floating-point error in the FLS polynomial means the raw result has residuals of $O(10^{-3})$; NS then converges to machine precision in a few iterations. `euclidean_inverse` does this automatically when `refine=True`.

[1]:
```
from clif.context import Cl
from clif.multivector import Accum
from clif.inverse import euclidean_inverse, newton_schulz
import numpy as np
import matplotlib.pyplot as plt
```

1.1 Manual iteration trace

We reproduce what `euclidean_inverse(refine=True)` does internally: get the raw FLS result, then pass it to `newton_schulz` as the initial guess.

[2]:
```
import clif.context as ctx
import clif.sign_table as ST

def residual(A, X):
    """Max-norm residual ||e0 - A*X|| across all blades."""
    r = (A * X).Reg.copy()
    r[0] -= 1.0
    return float(np.max(np.abs(r)))

Cl(15)
A = Accum.random()

X_raw = euclidean_inverse(A, refine=False)
```

```
print(f"Raw FLS residual at Cl(15): {residual(A, X_raw):.2e}")

X_refined = newton_schulz(A, X_raw)
print(f"After NS refinement:        {residual(A, X_refined):.2e}")
```

```
Raw FLS residual at Cl(15): 3.07e-06
err =  3.0686332235955256e-06
err =  1.7169489605899806e-09
After NS refinement:        9.21e-15
```

1.2 Convergence plot across dimensions

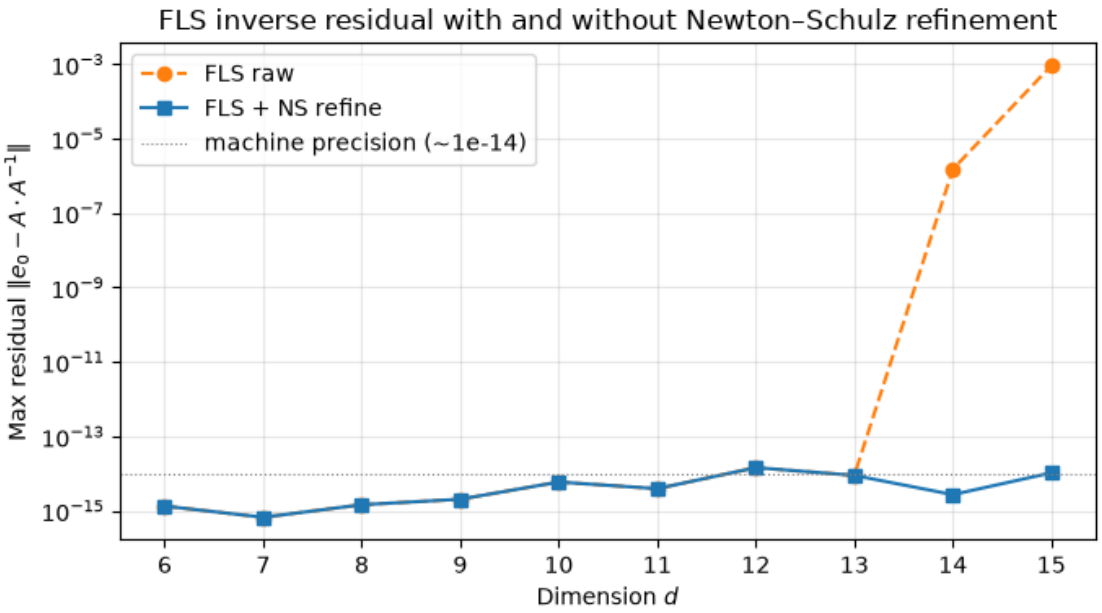
Raw FLS residual (before NS) vs dimension, showing where refinement becomes necessary.

[3]:
```
dims = list(range(6, 16))
raw_residuals = []
refined_residuals = []

for d in dims:
    Cl(d)
    A = Accum.random()
    X0 = euclidean_inverse(A, refine=False)
    Xr = euclidean_inverse(A, refine=True)
    raw_residuals.append(residual(A, X0))
    refined_residuals.append(residual(A, Xr))

fig, ax = plt.subplots(figsize=(7, 4))
ax.semilogy(dims, raw_residuals,     'o--', label='FLS raw',         color='tab:
 ↪orange')
ax.semilogy(dims, refined_residuals, 's-',  label='FLS + NS refine', color='tab:
 ↪blue')
ax.axhline(1e-14, color='gray', linestyle=':', linewidth=0.8, label='machine␣
 ↪precision (~1e-14)')
ax.set_xlabel('Dimension $d$')
ax.set_ylabel(r'Max residual $\|e_0 - A \cdot A^{-1}\|$')
ax.set_title('FLS inverse residual with and without Newton-Schulz refinement')
ax.set_xticks(dims)
ax.legend()
ax.grid(True, which='both', alpha=0.3)
plt.tight_layout()
plt.show()
```

```
err =  1.5068227508256956e-06
err =  1.335622723530605e-10
err =  0.0009132270050202921
err =  0.00010329284336403255
err =  8.763362179718627e-07
err =  4.172707063774053e-11
```



1.3 NS iteration steps at Cl(15)

Trace the residual at each Newton–Schulz step to show quadratic convergence.

[4]:
```
Cl(15)
A = Accum.random()
X0 = euclidean_inverse(A, refine=False)

# Re-implement iteration manually to collect per-step residuals
mul = ctx._ActiveTable.fast_mul
a = A.Reg
x = X0.Reg.copy()
two = np.zeros_like(a); two[0] = 2.0

steps = []
for _ in range(8):
    ax = mul(a, x)
    r = ax.copy(); r[0] -= 1.0
    err = float(np.max(np.abs(r)))
    steps.append(err)
    if err < 1e-14:
        break
    x = mul(x, two - ax)

print("Step  Residual")
for i, r in enumerate(steps):
```

```
    print(f"  {i}    {r:.2e}")
```

```
Step  Residual
  0    2.90e-08
  1    4.13e-14
  2    7.22e-15
```